

Step 2 — Database Persistence — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

db/models.py

```
from sqlalchemy import Column, String, DateTime
from sqlalchemy.orm import declarative_base

Base = declarative_base()

class Run(Base):
    __tablename__ = "runs"
    id = Column(String, primary_key=True)
    input = Column(String)
    output = Column(String, nullable=True)
    log = Column(String, nullable=True)
    status = Column(String, default="pending")
    created_at = Column(DateTime)
    completed_at = Column(DateTime, nullable=True)
```

db/database.py

```
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from db.models import Base

engine = create_engine("sqlite:///runs.db")
SessionLocal = sessionmaker(bind=engine)
Base.metadata.create_all(engine)
```

api/routes.py (excerpt)

```
from uuid import uuid4
from datetime import datetime, timezone
```

```

from fastapi import APIRouter, HTTPException
from db.database import SessionLocal
from db.models import Run

router = APIRouter()

@router.post("/run-task")
def run_task(request: TaskRequest):
    run_id = str(uuid4())
    session = SessionLocal()
    session.add(Run(id=run_id, input=request.topic, status="pending", created_at=
    session.commit()

    result = graph.invoke(make_initial_state(request.topic))

    run = session.get(Run, run_id)
    run.output = result["final_report"]
    run.status = "success"
    run.completed_at = datetime.now(timezone.utc)
    session.commit()
    session.close()

    return {"final_report": result["final_report"], "approved": result["review_ve

@router.get("/runs/{run_id}")
def get_run(run_id: str):
    session = SessionLocal()
    run = session.get(Run, run_id)
    session.close()
    if not run:
        raise HTTPException(status_code=404, detail="Run not found")
    return run

```

This works — every run gets saved and can be looked up by ID. The session is opened and closed by hand rather than with a context manager, so a mid-request exception leaves it open (a real connection leak). Nothing catches a database failure — it would surface as a raw `500` traceback, exactly what the Advanced test in Hint 1 is designed to catch.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — context-managed sessions, request ID threaded through logging

db/models.py

```
from datetime import datetime
from sqlalchemy import Column, String, DateTime
from sqlalchemy.orm import declarative_base

Base = declarative_base()

class Run(Base):
    __tablename__ = "runs"
    id: str = Column(String, primary_key=True)
    input: str = Column(String, nullable=False)
    output: str | None = Column(String, nullable=True)
    log: str | None = Column(String, nullable=True)
    status: str = Column(String, nullable=False, default="pending")
    created_at: datetime = Column(DateTime, nullable=False)
    completed_at: datetime | None = Column(DateTime, nullable=True)
```

api/routes.py

```
import logging
from uuid import uuid4
from datetime import datetime, timezone
from fastapi import APIRouter, HTTPException
from db.database import SessionLocal
from db.models import Run
from graph import graph
from state import make_initial_state
from api.schemas import TaskRequest, TaskResponse

logger = logging.getLogger("contentforge.api")
router = APIRouter()

@router.post("/run-task", response_model=TaskResponse)
```

```
def run_task(request: TaskRequest) -> TaskResponse:
    run_id = str(uuid4())
    logger.info("run %s starting for topic=%s", run_id, request.topic)

    with SessionLocal() as session:
        session.add(Run(id=run_id, input=request.topic, status="pending", created=
        session.commit()

    result = graph.invoke(make_initial_state(request.topic))
    logger.info("run %s graph finished, approved=%s", run_id, result["review_verd

    with SessionLocal() as session:
        run = session.get(Run, run_id)
        run.output = result["final_report"]
        run.status = "success"
        run.completed_at = datetime.now(timezone.utc)
        session.commit()

    return TaskResponse(final_report=result["final_report"], approved=result["rev

@router.get("/runs/{run_id}")
def get_run(run_id: str) -> Run:
    with SessionLocal() as session:
        run = session.get(Run, run_id)
    if not run:
        raise HTTPException(status_code=404, detail="Run not found")
    return run
```

Difference from Basic: `with SessionLocal() as session:` guarantees the session is closed even if something inside the block raises — Basic's manual `session.close()` never runs if an earlier line throws. `run_id` is generated once, at the top, and appears in every `logger.info(...)` call for this request, so every log line for one run is greppable by that ID afterward. Full type hints. Still missing: what happens to the client, and to the `Run` row's status, if the database write or the graph call itself raises — right now, either one crashes the whole request with an unhandled exception.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — every database call and the graph call independently guarded

```
import logging
from uuid import uuid4
from datetime import datetime, timezone
from fastapi import APIRouter, HTTPException
from db.database import SessionLocal
from db.models import Run
from graph import graph
from state import make_initial_state
from api.schemas import TaskRequest, TaskResponse

logger = logging.getLogger("contentforge.api")
router = APIRouter()

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest) -> TaskResponse:
    run_id = str(uuid4())
    logger.info("run %s starting for topic=%s", run_id, request.topic)

    try:
        with SessionLocal() as session:
            session.add(Run(id=run_id, input=request.topic, status="pending", created_at=datetime.now(timezone.utc)))
            session.commit()
    except Exception:
        logger.exception("run %s: failed to write initial pending row", run_id)
        raise HTTPException(status_code=500, detail="Could not start task – please try again")

    try:
        result = graph.invoke(make_initial_state(request.topic))
    except Exception:
        logger.exception("run %s: graph execution failed", run_id)
        try:
            with SessionLocal() as session:
                run = session.get(Run, run_id)
                run.status = "failed"
                run.completed_at = datetime.now(timezone.utc)
                session.commit()
```

```

    except Exception:
        logger.exception("run %s: also failed to record 'failed' status", run_id)
        raise HTTPException(status_code=500, detail="Task failed – please try again")

    try:
        with SessionLocal() as session:
            run = session.get(Run, run_id)
            run.output = result["final_report"]
            run.status = "success"
            run.completed_at = datetime.now(timezone.utc)
            session.commit()
    except Exception:
        logger.exception("run %s: graph succeeded but failed to save the result", run_id)
        raise HTTPException(status_code=500, detail="Task completed but could not save result")

    return TaskResponse(final_report=result["final_report"], approved=result["approved"])

@router.get("/runs/{run_id}")
def get_run(run_id: str) -> Run:
    try:
        with SessionLocal() as session:
            run = session.get(Run, run_id)
    except Exception:
        logger.exception("failed to look up run %s", run_id)
        raise HTTPException(status_code=500, detail="Could not look up run – please try again")

    if not run:
        raise HTTPException(status_code=404, detail="Run not found")

    return run

```

Expected behavior with the database connection killed mid-request (Doc12's Break-It exercise): whichever of the 3 guarded blocks was running when the connection died raises inside its own `try`, gets logged with `logger.exception(...)` (full traceback, server-side only), and the client receives a plain `{"detail": "..."}` message with a `500` — never a raw traceback.

Expected behavior calling `GET /runs/{id}` while that same run is still mid-flight: a `200` with `status: "pending"`, not a `404` — the row already exists because it was written before the graph ever ran.

Approach 2 — same guarantees, expressed as one helper instead of 3 repeated `try/except` blocks

```

from contextlib import contextmanager

@contextmanager

```

```

def db_session_or_500(run_id: str, action: str):
    try:
        with SessionLocal() as session:
            yield session
    except Exception:
        logger.exception("run %s: database error during %s", run_id, action)
        raise HTTPException(status_code=500, detail=f"Could not {action} – please

@router.post("/run-task", response_model=TaskResponse)
def run_task(request: TaskRequest) -> TaskResponse:
    run_id = str(uuid4())

    with db_session_or_500(run_id, "start task") as session:
        session.add(Run(id=run_id, input=request.topic, status="pending", created
        session.commit()

    try:
        result = graph.invoke(make_initial_state(request.topic))
    except Exception:
        logger.exception("run %s: graph execution failed", run_id)
        with db_session_or_500(run_id, "record failure"):
            pass # swallow secondary failure intentionally; already logged above
            raise HTTPException(status_code=500, detail="Task failed – please try aga

    with db_session_or_500(run_id, "save result") as session:
        run = session.get(Run, run_id)
        run.output = result["final_report"]
        run.status = "success"
        run.completed_at = datetime.now(timezone.utc)
        session.commit()

    return TaskResponse(final_report=result["final_report"], approved=result["rev

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate assumes every database call and the graph call all succeed — any one of them failing crashes the whole request with an unhandled exception and (depending on your FastAPI error-handling setup) potentially a raw traceback back to the client. Both Advanced approaches add the same real behavior: a "pending" row written before the graph runs (so `GET /runs/{id}` is honest about in-progress work), independent failure handling at each of the 3 database touch-points, and a "failed" status recorded when the graph itself blows up. Approach 1 spells out each `try/except` explicitly — more repetitive, but every failure path is visible in one read-through. Approach 2 extracts the repeated shape into a `db_session_or_500` context manager — less repetition, at the cost of the actual error handling now living in one shared helper instead of next to each call site.

Which one should you actually write? Approach 1 while this route is this simple — 3 database touch-points is not enough repetition yet to justify the indirection of a shared context manager, and explicit code here is easier for anyone (including future-you) to read top to bottom during an incident. Switch to Approach 2's pattern once you have several routes doing the same kind of guarded database work (Step 4 will add more) — that's the point where the helper starts paying for the extra abstraction it costs to understand.